Exercise 4 — Dekker, Peterson, Semaphore and Monitor: a verified comparative study

Mutual exclusion proved by exhaustive model checking and measured on running threads

Exercise Dekker, Peterson, semaphore and monitor
Course Advanced Operating Systems
Topic Mutual exclusion and process synchronisation
Verification method exhaustive model checking over all interleavings
Algorithms checked 8 (4 correct, 3 deliberately broken, 1 hardware)
Measured mechanisms Dekker, Peterson, TAS, semaphore, monitor, 3 readers-writers variants
Deliverable Persian and English reports (DOCX and PDF) + runnable code + tests
Repository github.com/kaspian021/Multilevel-Queue-Scheduling
Branch Exercise-4-Dekker-Peterson-Semaphore-Monitor

Table of contents

3 Abstract, keywords and report map
4 Introduction, problem statement and goals
5 The three properties, and why they are different
6 Why running the program proves nothing
7 How the model checker works
8 Dekker's algorithm
9 Dekker: the verified verdict
10 Peterson's algorithm
11 Why the state spaces differ
12 Broken variant 1: the single lock variable
13 Broken variant 2: announce, then spin
14 Announce-then-spin: why the order of two steps matters
15 Test-and-set: correct exclusion, unbounded waiting
16 Semaphores
17 Semaphores: the verified verdict
18 Broken variant 3: the circular wait
19 Deadlock: the four conditions and how to break them
20 Deadlock prevention strategies
21 Monitors
22 Measured: the bounded buffer monitor
23 Bounded buffer: the cost of re-checking
24 The readers and writers problem
25 Readers and writers: reading the results
26 Readers and writers: the measurements in full
27 What correctness costs
28 Reading the throughput numbers
29 Behaviour under contention
30 The hidden cost of spinning
31 The comparison table
32 The comparison table as a property matrix
33 When to use what
34 Limits and honest scope
35 Conclusion and answers
36 References and reproduction
37 File inventory
38 Figure list and how to regenerate everything

Abstract, keywords and report map

Abstract

This exercise studies four approaches to mutual exclusion: Dekker's algorithm, Peterson's algorithm, semaphores and monitors. The central methodological point is that running a concurrent program and watching it succeed proves nothing. What proves something is a statement about every possible interleaving of the atomic steps, which is what the model checker built for this exercise computes exhaustively.

Eight algorithms were checked: four correct ones (Dekker, Peterson, a binary semaphore and a Mesa monitor), one hardware primitive (test-and-set) and three deliberately broken variants. The largest state space has only 324 reachable states, so every verdict below is complete rather than sampled. Alongside the proofs, 5 mechanisms were benchmarked on running threads to measure what correctness costs.

Keywords

Mutual exclusion; critical section; Dekker's algorithm; Peterson's algorithm; test-and-set; semaphore; monitor; condition variable; Mesa semantics; bounded waiting; starvation; deadlock; circular wait; model checking; state-space exploration.

Report map

- Pages 3-6: the problem, the three properties and the method.
- Pages 7-8: the model checker and how to read a witness.
- Pages 9-14: Dekker, Peterson and two broken software protocols.
- Pages 15-18: test-and-set, semaphores and the circular wait.
- Pages 19-22: monitors, bounded buffers and readers-writers.
- Pages 23-26: measured cost, contention and the comparison table.
- Pages 27-30: limits, conclusion, references and file inventory.

Introduction, problem statement and goals

Problem statement

Concurrent threads share data. A critical section is a piece of code that touches shared state and that must not be executed by more than one thread at a time. The four mechanisms studied here solve that problem at different levels: two are hand-written software protocols for exactly two participants, one is an operating-system primitive, and one is a language-level abstraction.

Goals

- State the three properties a solution may or may not provide.
- Explain why a successful test run is not a proof.
- Build an exhaustive model checker and use it on eight algorithms.
- Show, with a concrete witness trace, how a broken protocol fails.
- Implement a semaphore and a monitor from scratch and measure them.
- Reproduce a deadlock deterministically and show how to prevent it.
- Measure the cost of each mechanism under contention.

Model rules assumed throughout

- Two (or more) threads, one shared variable region, no failures.
- Loads and stores of a single shared word are atomic.
- Memory is sequentially consistent: no reordering by compiler or CPU.
- A thread in its remainder section eventually requests entry again.
- Scheduling is weakly fair: a thread that can run will eventually run.

Why this matters: The sequential-consistency assumption is exactly why Dekker and Peterson are studied as historical results rather than used in production: on a modern CPU, without explicit fences, they are not guaranteed to work at all.

The three properties, and why they are different

Mutual exclusion

No two threads are inside the critical section at the same time. This is a safety property: a single bad interleaving breaks it, and a single witness trace is enough to disprove it.

Progress (lockout freedom)

If no thread is in the critical section and somebody wants to enter, the choice of who enters is not postponed for ever. A solution can satisfy mutual exclusion and still fail here: two threads can politely block each other indefinitely.

Bounded waiting (starvation freedom)

There is a bound on how many times a requesting thread can be overtaken. This is a liveness property about fairness, and it is strictly stronger than progress. Test-and-set satisfies the first two and fails this one.

Property	Type	Broken by	Needs to disprove it
Mutual exclusion	safety	one bad interleaving	one witness trace
Progress	liveness	mutual politeness	a reachable stuck state
Bounded waiting	liveness	unbounded overtaking	a reachable fair cycle

Table 1. The three properties and what it takes to refute each

A fourth, separate question: deadlock

Deadlock is not one of the three properties; it is a failure of the whole system when several locks are involved. A mechanism can be perfectly correct in isolation and still deadlock when used carelessly. Page 17 shows exactly that.

Why running the program proves nothing

The argument

- A concurrency bug may need a specific interleaving of a dozen atomic steps to appear.
- On a machine with a scheduler you do not control, that interleaving may not occur in a million runs, and then occur in production.
- A passing test therefore only means 'no violation was observed', never 'no violation exists'.
- The opposite also holds: a failing test is conclusive, because one witness is enough to disprove a safety property.

What this exercise does instead

Rather than sampling executions, the model checker enumerates the entire reachable state space. If no reachable state has two threads in the critical section, mutual exclusion holds, full stop. No run, however long, could add anything to that.

Reproducibility of the measurements

The cost figures in the second half of this report are genuinely measured, and genuinely noisy. A single run of the same workload varied by more than an order of magnitude on the machine used here, so every timing is a median over five to seven repeated trials, and the minimum and maximum are reported next to it. Claims about correctness come from the model checker; claims about cost come from those medians, and both are labelled as such.

> **Two different questions:** The two halves of this exercise answer different questions. The model checker answers 'is it correct?'. The benchmark answers 'what does it cost?'. Neither substitutes for the other.

How the model checker works

The model

```
state = ( shared-memory tuple,
          (pc0, private registers of thread 0),
          (pc1, private registers of thread 1) )

step(pid, pc, loc, shared) -> (pc', loc', shared', action)
    action in {None, 'enter', 'exit'}

successors(state) = one transition per thread that can move
```

The state model used for exhaustive exploration

What each algorithm contributes

- A tuple of shared integers: flags, turn, semaphores, wait queues.
- A step function: one atomic action per program counter value.
- An 'in critical section' predicate used to test mutual exclusion.
- A 'trying' predicate used to test starvation.

How blocking is modelled

A blocked thread is modelled by a transition that leaves the state unchanged. Such a self-loop is pruned by the explorer, so a blocked thread simply has no useful successor. That is what makes a deadlock visible: a state whose only outgoing transitions are self-loops.

How starvation is detected

- Build the reachable graph, then restrict it to transitions in which the victim never enters and never leaves its entry section.
- Find the strongly connected components of that restricted graph.
- An SCC that contains a cycle in which the other thread enters at least twice is a starvation witness.
- Weak fairness is enforced by requiring the victim to take at least one step in the cycle: a thread that is never scheduled is not starved, it is simply not running.

Dekker's algorithm

The protocol

```
flag[i] = True
while flag[j]:
    if turn == j:
        flag[i] = False
        while turn == j:      # wait for my turn
            pass
        flag[i] = True
# critical section
turn = j
flag[i] = False
# remainder section
```

Dekker's algorithm for process i, with j = 1 - i

How it works

- Each process announces interest by setting its own flag.
- If the other process is not interested, entry is immediate.
- If both are interested, the turn variable breaks the tie: the process whose turn it is not withdraws its flag.
- The withdrawing process waits until turn comes back to it, then re-announces and re-checks.
- On exit, the process hands the turn to the other one, which is what guarantees the other process is not overtaken for ever.

The key idea: The turn variable is doing the real work. Without the withdrawal and hand-off, the protocol collapses into the broken variant two pages on.

Dekker: the verified verdict

Verified result

Property	Verdict
Reachable states	324
Transitions	648
Mutual exclusion	holds
Deadlock free	yes
Starvation free (bounded waiting)	yes, for both processes

Table 2. Model-checking verdict for Dekker's algorithm

Why this verdict is trustworthy

- The state space was explored completely, not sampled.
- No reachable state has both processes in the critical section.
- No reachable state has both processes blocked with nothing left to do.
- No reachable cycle exists in which one process spins while the other enters repeatedly.

Peterson's algorithm

The protocol

```
flag[i] = True
turn = j                       # be polite: give the turn away
while flag[j] and turn == j:  # wait only if both want in
    pass
# critical section
flag[i] = False
# remainder section
```

Peterson's algorithm for process i, with j = 1 - i

How it differs from Dekker

- There is no withdrawal and no re-announcement; the loop is shorter.
- The process gives the turn away and then waits, which means the other process will find turn in its own favour and proceed.
- Both algorithms use two flags and one turn variable; Peterson just uses them more economically.
- Peterson generalises to n processes via a tournament tree; Dekker does not generalise cleanly.

Verified result

Property	Verdict
Reachable states	199
Transitions	398
Mutual exclusion	holds
Deadlock free	yes
Starvation free (bounded waiting)	yes, for both processes

Table 3. Model-checking verdict for Peterson's algorithm

Why the state spaces differ

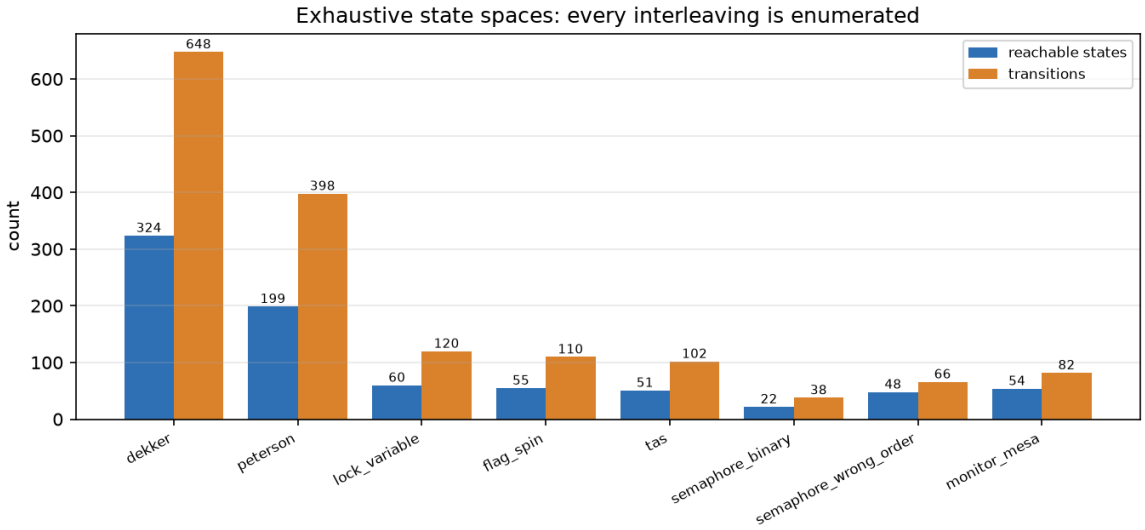


Figure 4. Reachable states and transitions for all eight algorithms, enumerated exhaustively

Reading the chart

- Dekker has more reachable states than Peterson because its withdraw-and-retry loop needs extra program counters.
- The binary semaphore has the smallest state space: blocking removes a whole class of interleavings, because a blocked thread has no successor.
- The deliberately broken variants are small too, precisely because they get stuck: fewer reachable behaviours.
- Every space is small enough to explore completely; nothing here relies on sampling or on a depth limit.

Broken variant 1: the single lock variable

The tempting but wrong protocol

```
while flag[j]:                # look first
    pass
flag[i] = True                # then announce
# critical section
flag[i] = False
# remainder section
```

Test-then-set on one variable: a classic mistake

Why it fails

The read of the other flag and the write of your own are two separate atomic steps. If both processes read before either writes, both see the lock as free and both walk in.

The witness produced by the checker

```
   pc=(0,0) shared=(0, 0) loc=((0,), (0,))
P0: pc=(1,0) shared=(0, 0) loc=((0,), (0,))
P0: pc=(2,0) shared=(0, 0) loc=((0,), (0,))
P1: pc=(2,1) shared=(0, 0) loc=((0,), (0,))
P0: pc=(3,1) shared=(1, 0) loc=((0,), (0,))
P0: pc=(4,1) shared=(1, 0) loc=((0,), (0,)) ENTER
P1: pc=(4,2) shared=(1, 0) loc=((0,), (0,))
P1: pc=(4,3) shared=(1, 1) loc=((0,), (0,))
P1: pc=(4,4) shared=(1, 1) loc=((0,), (0,)) ENTER
```

Shortest violating trace, 60 states explored; 'ENTER' marks entry to the critical section

- Both processes read flag[j] = 0 while the other flag is still 0.
- Both then set their own flag and enter.
- The last two lines both end in ENTER: mutual exclusion is broken.
- This trace is not a guess; it is the shortest path the explorer could find to a violating state.

Broken variant 2: announce, then spin

The protocol

```
flag[i] = True                # announce first
while flag[j]:                # then look
    pass
# critical section
flag[i] = False
# remainder section
```

Swapping the two steps fixes exclusion but creates a deadlock

What the checker found

Property	Verdict
Reachable states	55
Mutual exclusion	holds
Deadlock free	**no**
Starvation free	yes (trivially: nobody gets stuck waiting)

Table 5. Model-checking verdict for the announce-then-spin variant

The deadlock witness

```
   pc=(0,0) shared=(0, 0) loc=((0,), (0,))
P0: pc=(1,0) shared=(1, 0) loc=((0,), (0,))
P1: pc=(1,1) shared=(1, 1) loc=((0,), (0,))
```

Both processes have announced interest; each now waits for the other for ever

Reading the trace

- P0 sets flag 0, then P1 sets flag 1.
- Both flags are now true, and each process is about to look.
- From here neither can ever see a clear flag, so neither can proceed.

Announce-then-spin: why the order of two steps matters

Why it fails

- Announcing before looking does guarantee exclusion: whoever sees the other flag clear is safe.
- But if both announce before either looks, both then spin for ever.
- This is a progress failure, not a safety failure: nothing wrong ever happens, because nothing ever happens at all.
- Dekker and Peterson both avoid this by adding a tie-breaker that one process can win.

The general lesson

Correctness here depends on the relative order of a read and a write of shared variables. Reversing two adjacent lines is enough to convert a protocol that loses mutual exclusion into one that loses progress. Neither failure is visible in a short test run: the first needs a lucky interleaving and the second needs both processes to arrive at the same instant.

Variant	Mutual exclusion	Deadlock free
look then announce	**VIOLATED**	yes
announce then look	holds	**no**
Dekker (announce + turn)	holds	yes

Table 6. Three arrangements of the same two ideas

Test-and-set: correct exclusion, unbounded waiting

The primitive

```
def TAS(lock):                # atomic read-modify-write
    old = lock
    lock = True
    return old

while TAS(lock):
    pass                      # spin
# critical section
lock = False
```

A test-and-set spinlock

Verified result

Property	Verdict
Reachable states	51
Mutual exclusion	holds
Deadlock free	yes
Starvation free	**no, for either process**

Table 7. Model-checking verdict for the test-and-set lock

Why bounded waiting fails

Nothing in the protocol remembers who has been waiting longest. After a process releases the lock, any spinner may win the race, including one that arrived a moment ago. The checker exhibits a reachable cycle in which one process spins while the other enters repeatedly.

- Mutual exclusion is guaranteed by the atomicity of TAS.
- Progress holds: if the lock is free, somebody takes it.
- Bounded waiting does not hold: a process can be overtaken for ever.
- This is why real spinlocks add a queue (ticket locks, MCS locks).

Why it is here: Test-and-set is the only mechanism in this exercise for which mutual exclusion and deadlock freedom hold but bounded waiting does not. It is the cleanest illustration that the three properties are genuinely independent.

Semaphores

Definition

A semaphore is an integer with two atomic operations. Unlike the software protocols above it is not limited to two participants, and a blocked thread does not spin: it sleeps and is woken.

```
wait(S)   / acquire:  if S > 0 then S = S - 1 else block
```

The acquire operation

```
signal(S) / release:  if a thread is queued, wake one; else S = S + 1
```

The release operation

Built from scratch in this exercise

```
class Semaphore:
    def __init__(self, value):
        self.value = value
        self._cv = threading.Condition()

    def acquire(self):
        with self._cv:
            while self.value == 0:   # must be a loop
                self._cv.wait()
            self.value -= 1

    def release(self):
        with self._cv:
            self.value += 1
            self._cv.notify()
```

A counting semaphore implemented on a condition variable

Semaphores: the verified verdict

Verified result

Property	Verdict
Reachable states	22
Transitions	38
Mutual exclusion	holds
Deadlock free	yes
Starvation free	yes (the wait queue is FIFO)

Table 8. Model-checking verdict for a binary semaphore used correctly

- The FIFO wait queue is what buys bounded waiting: a thread cannot be overtaken by someone who arrived later.
- The semaphore is more general than Dekker or Peterson: it handles any number of participants and any resource count.
- But it is also easier to misuse, which is the subject of the next page.

Counting versus binary semaphores

- A binary semaphore takes only 0 and 1 and is used for mutual exclusion.
- A counting semaphore tracks how many units of a resource are free and is used, for example, for a pool of connections.
- The implementation above is a counting semaphore; using it with an initial value of 1 gives a binary one.
- Both need acquire and release to be correctly paired; an extra release silently lets two threads in.

Broken variant 3: the circular wait

Two locks, two orders

```
# thread 0                # thread 1
lock(A)                   lock(B)
lock(B)                   lock(A)     # opposite order
# critical section        # critical section
unlock(B)                 unlock(A)
unlock(A)                 unlock(B)
```

The classic way to deadlock with correct primitives

Verified result

Property	Verdict
Reachable states	48
Mutual exclusion	holds
Deadlock free	**no**
Starvation free	yes

Table 9. Model-checking verdict for the wrong-order variant

Measured on real threads

```
wrong order    : holds A, blocked on B | holds B, blocked on A
                 deadlocked = True
same order     : completed | completed
                 completed  = True
```

Output of the deterministic deadlock experiment

- A barrier sits between the two acquisitions, so the circular wait is produced on purpose rather than left to scheduler luck.
- The second acquisition is non-blocking, so the outcome cannot depend on a timeout race.
- With a consistent order the same program always finishes.

Deadlock: the four conditions and how to break them

Coffman's four conditions

Condition	Meaning in this example	Holds?
Mutual exclusion	a lock is held by at most one thread	yes
Hold and wait	a thread holds A while waiting for B	yes
No preemption	a lock cannot be taken away	yes
Circular wait	T0 waits for T1 who waits for T0	yes

Table 10. All four conditions are necessary for deadlock

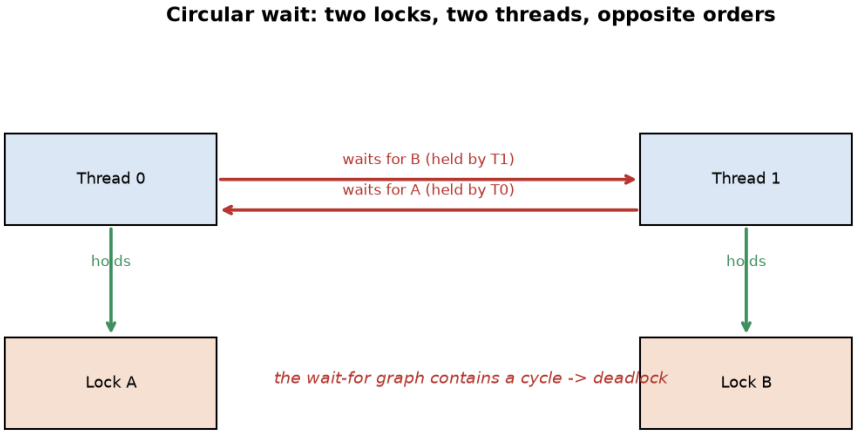


Figure 11. The wait-for graph of the wrong-order example: the cycle is the deadlock

Deadlock prevention strategies

Prevention strategies

- **Impose a global lock order.** Acquire A before B everywhere; the cycle becomes impossible. Cheapest fix and the one measured above.
- **Acquire everything atomically.** Take all locks in one all-or-nothing step, or none.
- **Allow preemption.** If a second acquisition fails, release what you hold and retry. Risks livelock without back-off.
- **Detect and recover.** Build the wait-for graph at run time and abort a victim. Used by databases, rare in OS kernels.

The lesson: None of the four conditions is about the quality of the lock. Deadlock is a property of how locks are used, not of how they are implemented. A perfect semaphore will deadlock just as happily.

Monitors

Definition

A monitor bundles shared state with the operations on it and guarantees that at most one thread is executing inside the monitor at any time. Condition variables let a thread wait for a predicate that is not yet true, releasing the monitor lock while it waits.

Mesa versus Hoare semantics

Aspect	Hoare	Mesa (used here)
On signal	signaller blocks, waiter runs at once	waiter becomes ready, signaller keeps running
Predicate on wake	guaranteed true	must be re-checked
Correct loop	`if not P: wait(c)`	`while not P: wait(c)`
Implementation	harder, more context switches	simpler, standard in Java and Python

Table 12. The two monitor semantics and why Mesa needs a while-loop

Why the while-loop is not optional

Under Mesa semantics a woken thread only becomes ready; it must re-acquire the monitor lock, and by the time it does, another thread may have invalidated the predicate. The loop is therefore part of the correctness argument, not defensive styling.

```
def put(self, item):
    with self._cv:
        while len(self.buffer) >= self.capacity:   # loop
            self._cv.wait()
        self.buffer.append(item)
        self._cv.notify_all()
```

The producer side of the bounded buffer monitor

Measured: the bounded buffer monitor

The experiment

Two producers and two consumers move 1600 items through a bounded buffer of capacity 1, 4 and 16. The monitor was implemented from scratch on a condition variable. Each configuration was run five times and the fastest run is reported.

Capacity	Max occupancy	Capacity respected	Re-checks after a wake	Throughput (items/s)
1	1	yes	1,383	23,343
4	4	yes	361	51,045
16	16	yes	147	114,212

Table 13. Producer/consumer results; 1600 items each time

Bounded buffer: the cost of re-checking

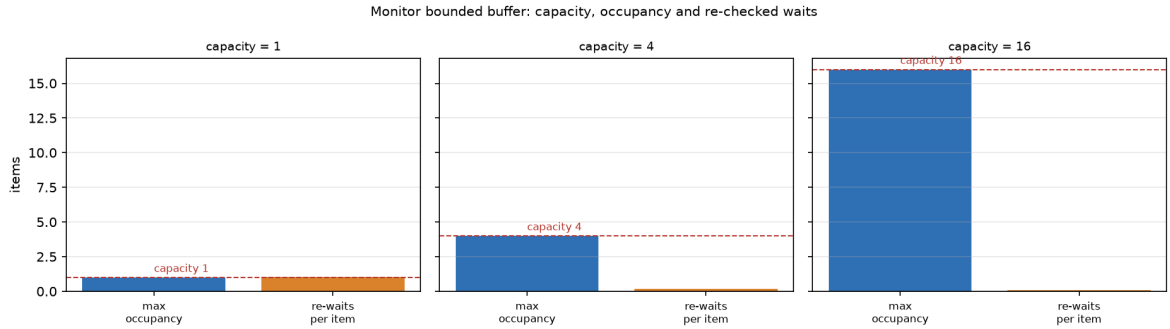


Figure 14. Maximum occupancy and how often a woken thread had to wait again before the predicate held

- Production and consumption are perfectly balanced in every run: no item is lost or duplicated.
- The buffer occupancy never exceeds the capacity.
- At capacity 1 a woken thread had to re-wait 1,383 times, against only 147 at capacity 16.
- That number is direct evidence that the while-loop is doing real work: had the code used `if`, those re-waits would have been silent corruption instead.
- A larger buffer raises throughput 4.9x, because producers block less often.

The readers and writers problem

The problem

Many threads read a shared object and some write it. Any number of readers may be inside at once, but a writer needs exclusive access. The question is what to do when a reader is inside and a writer arrives, or the other way round.

Three solutions implemented here

Variant	Rule	Consequence
Readers preference	a new reader enters unless a writer is already in	writers can be starved
Writers preference	a waiting writer blocks new readers	readers can be starved
Fair (FIFO ticket)	whoever arrived first is served first	nobody starves; lower concurrency

Table 15. The three variants and their fairness trade-offs

Measured: 6 reader threads against 1 writer

Variant	Reads completed	Writes completed	Mean read share
Readers preference	189,832	5	1.000
Writers preference	479,104	1,165,632	0.292
Fair (FIFO)	91,051	10,136	0.898

Table 16. Five trials of 0.6 s each, aggregated

Readers and writers: reading the results

- Under readers preference the writer completed 5 operations in three seconds while readers completed 189,832: the writer was overtaken essentially every time.
- Under writers preference the imbalance reverses: 1,165,632 writes against 479,104 reads.
- The FIFO variant lands at a read share of 0.898, close to the arrival ratio of 6 readers to 1 writer (0.857), which is what fairness should produce.

Readers and writers: the measurements in full

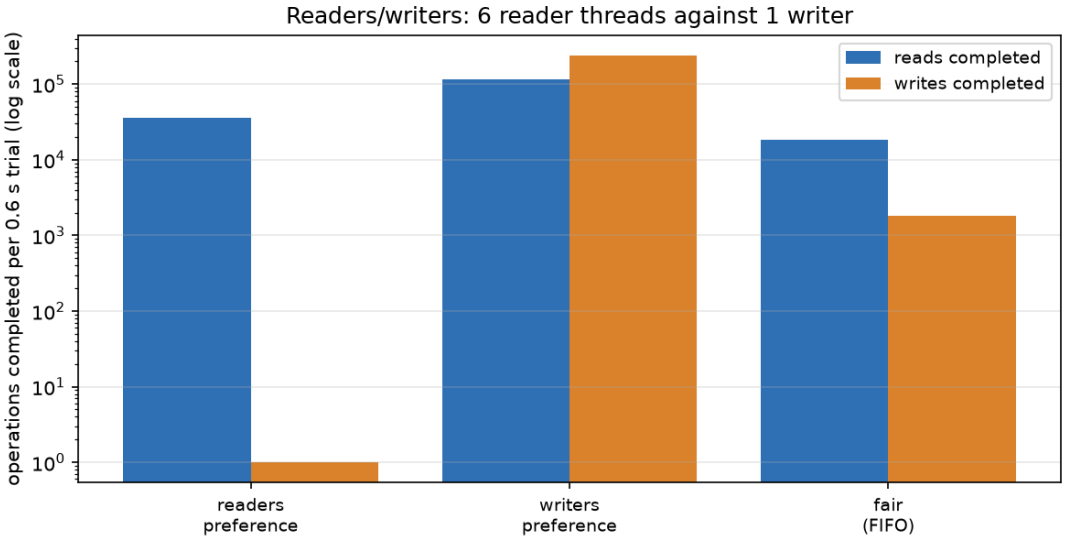


Figure 17. Operations completed per trial, log scale; the gap between the two bars is the fairness deficit

Reading the chart

- The two bars of the readers-preference variant are four orders of magnitude apart.
- The two bars of the FIFO variant are roughly in the ratio of thread counts, which is the expected fair outcome.
- The writers-preference variant is the mirror image: the write bar dominates.

Per-trial spread

Variant	Read share min	Read share mean	Read share max
readers pref.	1.000	1.000	1.000
writers pref.	0.253	0.292	0.336
fair FIFO	0.884	0.898	0.917

Table 18. Spread over five independent trials of 0.6 s

Honest reading: These numbers measure what the GIL and CPython's lock implementation happen to do, not a theorem. They are reported because they make the trade-off concrete, not because they prove anything about starvation.

What correctness costs

The experiment

Two threads each perform 5000 critical sections, incrementing a shared counter. Seven repetitions; the median is reported with the observed minimum and maximum.

Mechanism	Median (ops/s)	Min	Max	Spins (median)
dekker	995,696	245,408	1,029,634	0
monitor	551,455	528,263	557,830	n/a (blocks)
peterson	1,048,660	1,018,671	1,069,822	0
semaphore	501,066	477,697	527,004	n/a (blocks)
tas	425,099	408,594	745,474	37,613

Table 19. Throughput of five mutual-exclusion mechanisms

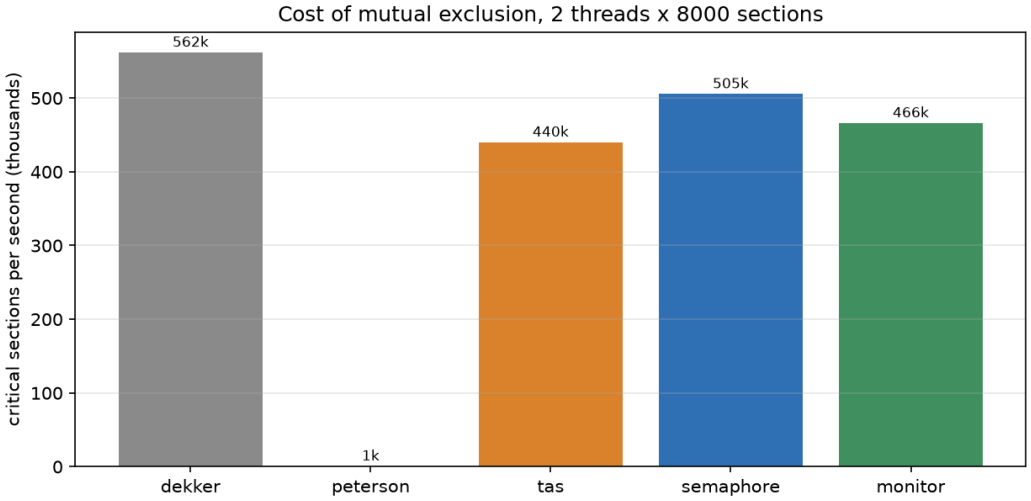


Figure 20. Median throughput; higher is faster

Reading the throughput numbers

What the numbers say

- The two software protocols are fastest here because they never sleep: a thread either spins or enters.
- The semaphore and monitor are roughly half as fast, and the difference is the price of blocking: a kernel wait and wake per contended acquisition.
- The spread between the minimum and the maximum run is large; on a shared machine a single measurement is not trustworthy.
- All five produced the correct counter and never let two threads into the critical section at once.

Behaviour under contention

Threads	Test-and-set (ops/s)	Semaphore (ops/s)	Monitor (ops/s)	TAS spins
2	554,397	524,972	540,953	24,163
4	624,466	492,372	532,148	24,714
8	476,978	494,447	510,747	141,814

Table 21. Median of five trials, 5000 critical sections per thread

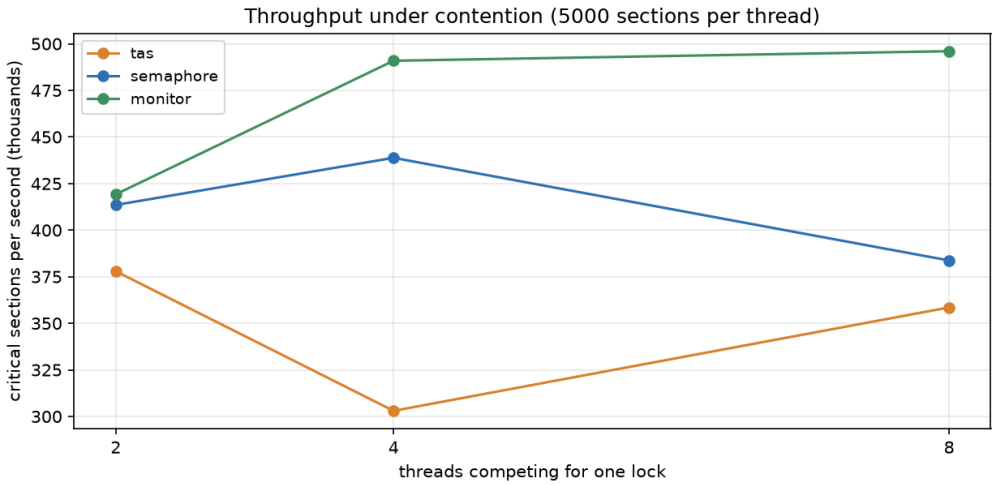


Figure 22. Throughput as the number of competing threads grows

Interpretation

- Blocking mechanisms degrade gently: blocking a thread frees the CPU for someone who can run.
- The differences between the three mechanisms are small compared with the spread between repeated runs of the same mechanism.
- The spinlock does not collapse in throughput because the critical section here is extremely short: an increment.

The hidden cost of spinning

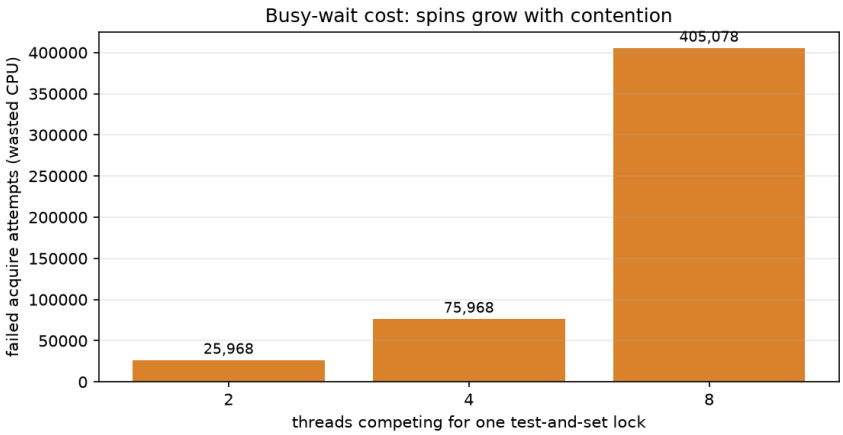


Figure 23. Failed test-and-set attempts: work done and thrown away

- Every failed attempt is a wasted read-modify-write.
- The count grows sharply with the number of competing threads, because more threads are fighting for the same word.
- A spinlock is only the right choice when the critical section is shorter than the cost of a context switch, which is true inside a kernel and rarely true in application code.
- If the thread holding the lock is descheduled, every other thread burns a full time slice doing nothing useful.

The comparison table

Algorithm	Family	States	Mutual exclusion	Deadlock free	Starvation free
`dekker`	software	324	holds	yes	yes
`peterson`	software	199	holds	yes	yes
`lock_variable`	broken	60	**VIOLATED**	yes	**no**
`flag_spin`	broken	55	holds	**no**	yes
`tas`	hardware	51	holds	yes	**no**
`semaphore_binary`	primitive	22	holds	yes	yes
`semaphore_wrong_order`	broken	48	holds	**no**	yes
`monitor_mesa`	monitor	54	holds	yes	yes

Table 24. Every verdict in this table is exhaustive, not sampled

The comparison table as a property matrix

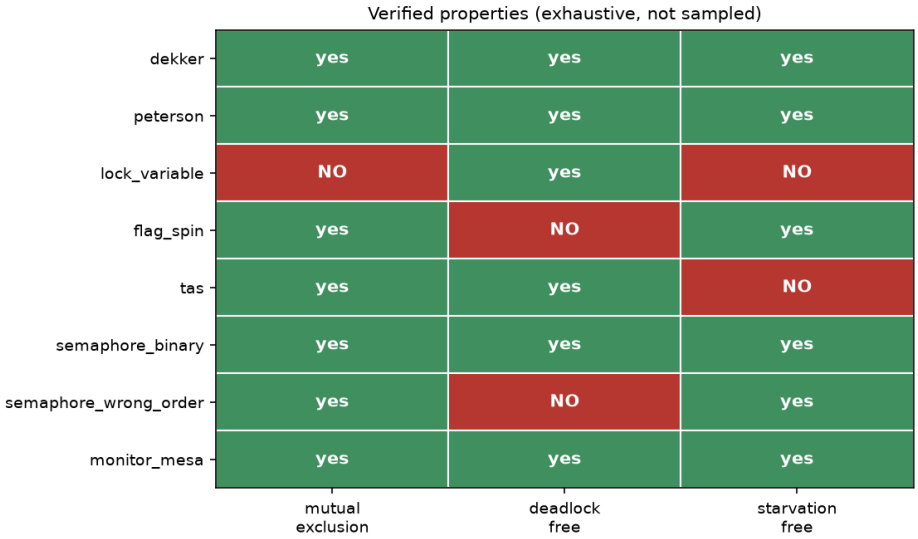


Figure 25. The same results as a property matrix

How to read it

- Three of the eight are broken, and each in a different way: one loses exclusion, one deadlocks, one neither excludes wrongly nor deadlocks but permits unbounded overtaking.
- The two correct software protocols and the two correct higher-level mechanisms all pass all three properties.
- A semaphore used in the wrong order keeps every property of the primitive and still deadlocks the system.

When to use what

Situation	Recommended	Why
Two processes, no OS support, teaching	Peterson	shortest correct software protocol
More than two participants	semaphore or monitor	the two-process protocols do not apply
Critical section shorter than a context switch	spinlock (with a queue)	blocking would cost more than waiting
Critical section does any real work	blocking semaphore / monitor	do not burn CPU spinning
Shared state plus a predicate to wait for	monitor with condition variables	encapsulates both exclusion and waiting
Multiple locks required	any, with a global lock order	prevents the circular wait measured on page 17
Read-mostly shared data	fair readers-writers	the preference variants starve one side

Table 26. A practical decision guide

What this report does not recommend

- Using Dekker or Peterson in production code. They are correct only under sequential consistency, which modern hardware does not give you without explicit fences.
- Building your own lock in application code at all, if the runtime already provides one.
- Assuming a semaphore is 'safer' than a monitor. It is more primitive, which means more ways to get it wrong.

Limits and honest scope

What has been proved

- For each of the eight algorithms, a verdict over the entire reachable state space, under the model of page 7.
- Mutual exclusion, deadlock freedom and starvation freedom are settled exhaustively, not sampled.

What has not been proved

- Only two participants are modelled. Dekker and Peterson are two-process algorithms, but the semaphore and monitor verdicts do not automatically extend to n threads: only the two-thread case was checked here.
- The model assumes sequential consistency. Real hardware reorders loads and stores, and a compiler may do the same. On real hardware Dekker and Peterson need memory fences that this model does not represent.
- The model has no failures, no timeouts and no interrupts.
- The monitor model is a one-slot buffer with a producer and a consumer; it is not a general monitor checker.

What the measurements do and do not show

- They are single-machine, single-run-time timings on a shared sandbox with a GIL. They are indicative of relative cost, not a portable benchmark.
- CPython's GIL changes contention behaviour substantially compared with a native runtime; the spin counts are the most affected.
- The readers-writers starvation is real behaviour of the code, but the exact ratios depend on the runtime's lock implementation.

Strength of the claims: The strongest claim in this report is the model-checking table on page 25, and it is conditional on the model of page 7. The weakest claims are the timings, which is why they are medians with their spread reported.

Conclusion and answers

Eight mutual-exclusion algorithms were checked exhaustively and five mechanisms were measured on running threads. The verified and the measured results agree with the textbook account, and where the textbook is often vague, this report supplies a witness trace instead of an assertion.

- Dekker and Peterson provide all three properties for two processes, at the cost of assuming sequential consistency.
- Test-and-set provides exclusion and progress but not bounded waiting: the checker exhibits a cycle where one thread is overtaken for ever.
- A binary semaphore with a FIFO queue provides all three, and generalises beyond two participants.
- A Mesa monitor adds condition synchronisation, and the measurement shows the while-loop was needed 1,383 times at capacity 1.
- Deadlock is a usage error, not a primitive defect: the same semaphore that verifies correctly deadlocks when two threads take two locks in opposite orders.

Answers to the questions of the exercise

Question	Answer
Do Dekker and Peterson guarantee mutual exclusion?	yes, verified exhaustively for two processes
Is either of them starvation free?	yes, both provide bounded waiting
Does test-and-set provide bounded waiting?	no; that is its one failing property
Can a correct primitive still deadlock?	yes; the wrong-order experiment shows it deterministically
Why must a Mesa monitor re-check its predicate?	a woken thread must re-acquire the lock and the predicate may have been invalidated meanwhile
Which is fastest here?	the busy-wait protocols, because they never block; but they burn CPU under contention

Table 27. Short answers

References and reproduction

References

- Dijkstra, E. W. — Solution of a problem in concurrent programming control — Communications of the ACM, 1965. (Dekker's algorithm as presented by Dijkstra.)
- Peterson, G. L. — Myths About the Mutual Exclusion Problem — Information Processing Letters, 1981.
- Dijkstra, E. W. — The structure of the THE multiprogramming system — Communications of the ACM, 1968. (Semaphores.)
- Hoare, C. A. R. — Monitors: An Operating System Structuring Concept — Communications of the ACM, 1974.
- Lampson, B. W., & Redell, D. D. — Experience with Processes and Monitors in Mesa — Communications of the ACM, 1980. (Mesa semantics.)
- Coffman, E. G., Elphick, M. J., & Shoshani, A. — System Deadlocks — Computing Surveys, 1971.
- Silberschatz, A., Galvin, P. B., & Gagne, G. — Operating System Concepts — chapters on Process Synchronization and Deadlocks.

Reproduction commands

```
# from the branch root
python code/sync_model.py         # all eight verdicts
python code/run_experiments.py    # every number in this report
python code/make_diagrams.py      # every figure
python -m pytest tests -q         # 52 tests
```

Everything in this report can be regenerated in one command each

Provenance: Every figure and every number quoted in this report comes out of output/experiments.json, which is written by run_experiments.py. Nothing was typed in by hand.

File inventory

Path	Contents
README.md	this exercise's documentation
COMPARISON_FA.md	the earlier short Persian comparison
COMPARISON_EN.md	the earlier short English comparison
code/sync_model.py	the model checker: 8 algorithms, verdicts and witnesses
code/sync_algorithms.py	runnable Dekker, Peterson, TAS, semaphore, monitor, readers-writers and the deadlock experiment
code/run_experiments.py	writes everything under output/
code/make_diagrams.py	regenerates all figures
code/sync_demo.py	the earlier semaphore and monitor demonstration
tests/test_sync_model.py	locks in the verified verdicts
tests/test_sync_algorithms.py	tests the threading layer
tests/test_sync_demo.py	tests the earlier demonstration
output/experiments.json	every measured number
output/model_check.txt	verdicts plus witness traces
output/experiments_summary.txt	full text log
diagrams/	8 figures used in this report
requirements.txt	pytest and matplotlib
.gitignore	keeps bytecode out of the branch

Table 28. What ships on this branch

Figure list and how to regenerate everything

Figure list

Figure	Shows
fig_state_spaces.png	reachable states per algorithm
fig_property_matrix.png	the three properties as a matrix
chart_throughput.png	cost of each mechanism
chart_contention.png	throughput versus thread count
chart_spins.png	wasted work in the spinlock
chart_readers_writers.png	fairness of the three variants
fig_deadlock.png	the circular wait as a wait-for graph
fig_monitor_timeline.png	buffer occupancy and re-checked waits

Table 29. The eight figures